



Savitribai Phule Pune University
Centre For Distance and Online Education

SAVITRIBAI PHULE PUNE UNIVERSITY, PUNE

CENTRE FOR DISTANCE AND ONLINE EDUCATION

Program	Master of Computer Application	
Course	MJ Programming from First Principles	
Topic Name	User defined types as a means to model (Practice Session)	
Topic Code	-	
Unit/Module No. & Name	10	User defined types
Self-Learning Hours	-	

Topic Details

S.No	Topic	Pg.No
1.0	Introduction	4
2.0	Meaning And Definition	5-7
3.0	Nature Or Type	8-10
4.0	Examples And Case Studies	11-13
5.0	Keywords And Touch Vocabulary	14



OBJECTIVE

1. A data type defines the kind of data a program can use and the operations allowed on that data.
2. Data types are more than labels because they shape how programs store, process, and protect information.
3. User-defined types help model complex real-world objects in a clear, structured, and safe way.
4. This unit connects basic programming knowledge with a formal understanding of the role of types.
5. By the end of this unit, you should be able to identify nouns in a problem statement that can become user-defined types.
6. You should understand why types like Student or Track are better than using unrelated individual variables.
7. You will be able to name and describe common user-defined types such as records and classes in simple, correct sentences.
8. You will learn to judge whether a type is too large or too small for the problem it represents.
9. You will notice when two concepts that should be separate are incorrectly mixed into one type.



1.0 INTRODUCTION

1.1 Why We Study Data Types

Every computer program works with some kind of data. A calculator uses numbers, a chat app uses messages and usernames, and a music app uses songs and playlists. When you write a program, you must tell the computer what type of data you want to store and how you plan to use it. This is what a data type does.

Most programming languages give you simple, basic types like integer, float, character, string, and Boolean. These are like small building blocks. They are useful, but they are not enough to represent real-life things. For example, a college student is not just a name. A student has an ID, courses, grades, phone number, and many other details.

User-defined types let you create your own custom building blocks. You can group related pieces of information and give them one name. The computer then treats this group as a new type—just like int or string. This means you create your own data types to match the real-world problem you are solving.

In beginner programming, students only use simple types at first. But as programs get bigger, it becomes confusing to handle many separate variables. Soon, you see that some variables always belong together. This is the stage where user-defined types become important. They help organize the program and make it easy to understand.

1.1.1 Real World And Programs

There is a strong connection between real life and computer programs. When we say modeling, we mean making a simple version of real things so the computer can work with them. A map is a model of a real city. In the same way, a user-defined type is a model of a real or imaginary thing inside a program.

You cannot put a real person inside a computer, but you can store important information about that person. You choose what information to keep and how to arrange it. If you do this carefully, your type becomes simple enough for the computer but useful enough for your problem. Good modeling means choosing what to include—and what to leave out.

2.0 MEANING AND DEFINITION

2.1 Definition Of Data Type

In computer science, a data type is a rule that tells us what kind of value we have and what operations we can perform on that value. An integer type allows addition and subtraction. A string type allows operations like joining two strings or searching for a letter. The type helps the compiler or interpreter check that you are using values in a safe and meaningful way.

We can think of a type as a label plus a set of allowed actions. When we declare a variable with a type, we are saying, This box will hold values of this shape, and I will use them in these ways. If we try to use the variable in a wrong way, the type system can warn us.

2.1.1 Primitive Types

Primitive types are built into the language. They are usually implemented in a direct way by the hardware of the computer. Examples include integer types of different sizes, floating point numbers, characters, and Boolean values. They are called primitive because they cannot be broken into smaller parts by the type system.

Even though primitive types are simple, they often come with details such as size limits, signed or unsigned modes, and memory layout. Programmers must understand these details when they work close to the hardware. But in most high level programming tasks, primitive types are treated as basic tools that just work.

2.1.2 User Defined Types

A user defined type is a data type that is created by the programmer, using language features such as structures, classes, records, or enumerations. While primitive types are supplied by the language designers, user defined types come from the needs of a specific problem.

We define a user defined type by giving it a name and describing its internal structure. The structure can be made from primitive types and from other user defined types. Once we have defined such a type, we can declare variables of that type, pass them to functions, return them from methods, and store them in collections.

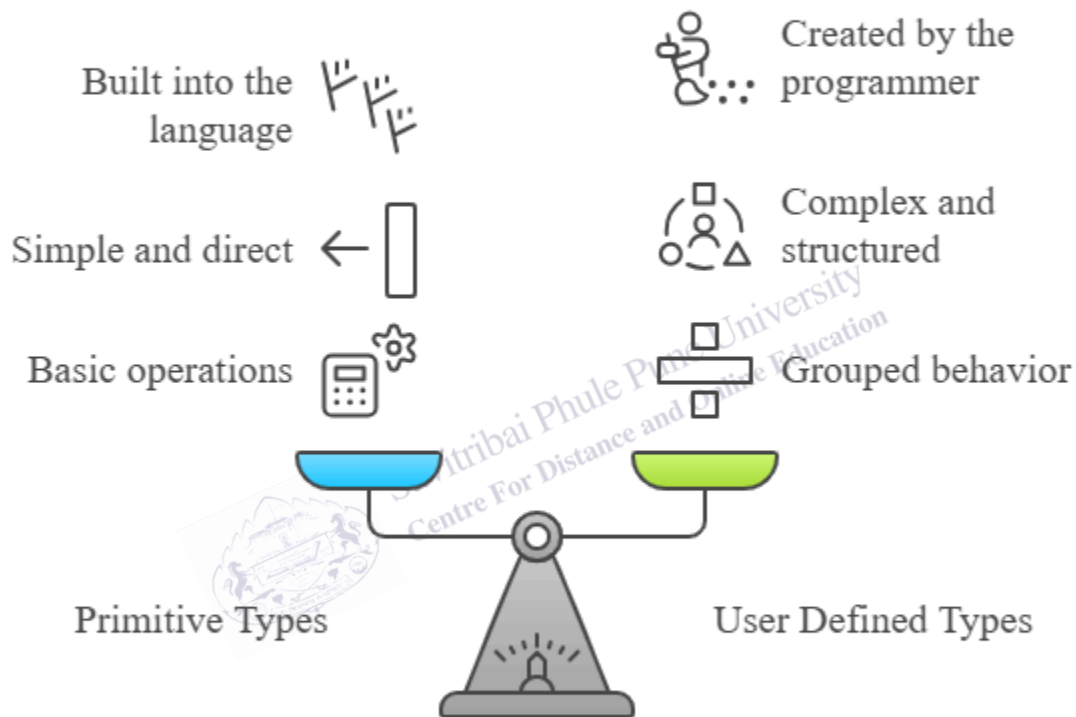
In many languages, user defined types also group behavior with data. For example, a Song class in an object oriented language may contain fields such as title, artist, and duration, plus methods such as play, pause, and display info. The type becomes a unit that holds both state and behavior.

2.2 Formal View Of User Defined Types

In a more formal view, a user defined type describes a set of possible values and a set of operations that make sense for those values. The values are built from other types using builders such as record of, list of, or either this or that. The operations may include reading and writing fields, comparing two values, or applying domain specific rules.

Fig. 1

Compare primitive and user-defined types in computer science.



This diagram compares primitive and user-defined data types. Primitive types are built into the language and handle simple values with basic operations, while user-defined types are created by programmers to model complex, structured data and grouped behavior.

Formal views matter when we reason about program correctness. If we know that a certain type always has a nonempty list of items, we can prove that some functions never face an empty input. If we know that a field is an enumeration with only three states, we can check that our code handles all three states.

2.3 Built In Versus User Defined Types

Built in types are general tools that work in almost every program. User defined types are custom tools that match one family of problems well. Built in types are usually few, stable, and highly tuned. User defined types can be many and flexible, and they change as the design of the system changes.

In practice, we combine both. Primitive types handle low level details, while user defined types handle higher level ideas such as Customer, Order, Playlist, or Appointment.

2.4 Role Of Type Systems

A type system is the set of rules that a language uses to connect types with values and expressions. Some type systems are static, meaning they check most rules before the program runs. Others are dynamic, meaning they check many rules while the program is running. Many modern languages mix both ideas.

For user defined types, the type system helps catch mismatches. If a function expects a Track and you pass a Patient, the compiler can stop you. If a field is declared as an enumeration and you try to store a number outside the allowed set, the system can complain.

2.5 Summary Of Meaning And Definition

The meaning and definition part of this unit shows that types are not random labels. A type is a rule about values and a promise about how those values can be used. Primitive types give us basic building blocks like numbers and characters. User defined types let us combine these blocks into shapes that match real world ideas, such as students, tracks, or appointments.

When you design a program, you are really choosing which types to use and how they relate to each other. Clear definitions at this stage make the later code easier to read, test, and change. In this way, the meaning and formal view of types guide the whole design.

3.0 NATURE OR TYPE

3.1 Kinds Of User Defined Types

User defined types appear in many forms, depending on the language and the style of programming. The most common forms are records or structs, classes and objects, enumerations, and unions or variant types. Each form has its own nature and is suited to certain modeling tasks.

When you design a system, it helps to know which form to choose. A record is simple and close to the data. A class adds behavior and hiding of details. An enumeration gives a short list of states. A variant lets a value take one of several shapes. Together, they form a small toolbox for modeling many different problems.

3.1.1 Records Or Structs

A record, often called a struct in languages like C, is a simple user defined type that groups several named fields. Each field has its own type. All fields are stored together in memory, often in a fixed layout. Records are good for modeling simple objects that mostly store data and do not need complex behavior.

For example, a record for a student might contain an integer field for the student id, a string field for the name, and another string field for the major. Functions that work on students can take a record as a parameter, but the record itself does not carry methods. This keeps the data and behavior separate.

3.1.2 Classes And Objects

A class is a more powerful user defined type that supports both data and methods in one unit. An object is a concrete instance of a class. Classes are central to object oriented programming languages. They allow ideas like encapsulation, which means hiding inner details, and inheritance, which means sharing behavior between related classes.

When we say that user defined types are a means to model real systems, classes are often what we imagine. A class can hide its internal fields and show only a clean interface. This gives other parts of the program a simple way to use the model without needing to know how it is built inside.

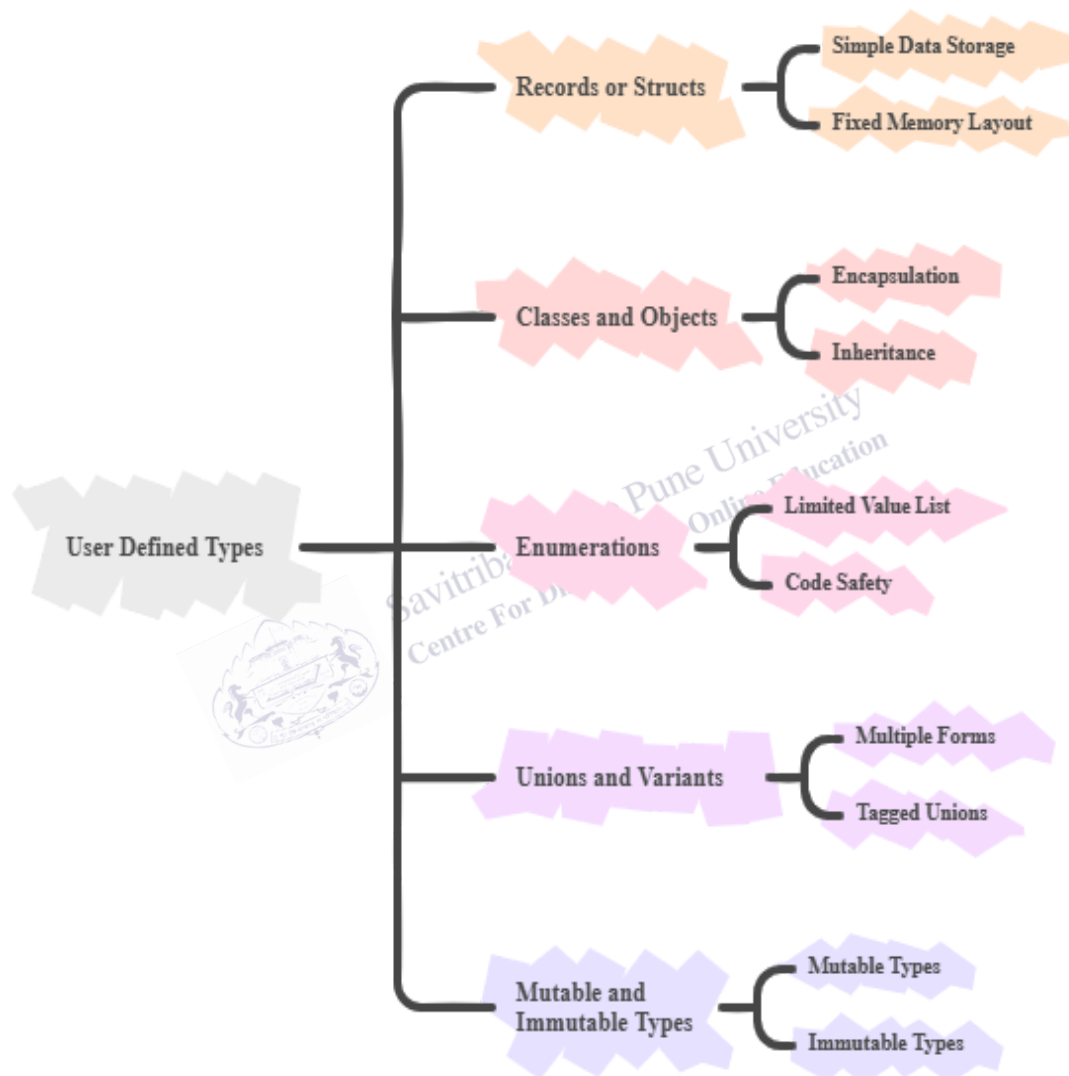
3.1.3 Enumerations

An enumeration, or enum, is a user defined type that lists a small set of named values. It is useful when a variable can only take one value from a limited list, such as days of the week, levels of access, or states of a process. Enums make code safer and clearer because they avoid magic numbers and repeated string names.

For example, instead of using 0, 1, and 2 to mean Stopped, Playing, and Paused, a music player program can define an enum called Player state with members STOPPED, PLAYING, and PAUSED. The compiler can then check that only these states are used in the code.

Fig. 2

User Defined Types in Programming



This diagram explains user-defined types in programming. It shows how programmers create structures, classes, enumerations, unions, and mutable or immutable types to organize data, control behavior, improve safety, and manage complexity in software systems.

3.1.4 Unions And Variants

A union or variant type allows a value to take one of several different shapes. At any time, the value has one active form, and the program must know which form is in use. Modern languages often provide safe variant types, sometimes called tagged unions, which carry both the data and a tag that says which case is active.

Unions are helpful when modeling data that can appear in several related forms. For example, a program that reads media files could define a variant type that can represent an MP3 track, a video clip, or a podcast episode, all under one umbrella type named Media item.

3.2 Choosing The Right User Defined Type

When you design a system, you must choose which form of user defined type fits each part of the model. If you only need to group a few fields, a record may be enough. If you need to attach rich behavior and hide internal details, a class may be better. If a value can only be one of a few named states, an enum is a clear choice. If a value may take several different shapes, a union or variant fits well.

Good designers think about how the types will be used. They avoid making a single giant type with too many fields, because such a type becomes hard to understand and maintain. Instead, they create smaller, focused types that each represent a clear idea.

3.3 Mutable And Immutable Types

Another part of the nature of a type is whether its values can change after creation. A mutable type allows its fields to be updated. An immutable type does not allow such changes; instead, any update creates a new value. Immutable types are common in functional programming and can reduce bugs related to unexpected changes.

For user defined types, you should decide which fields, if any, may change. For example, a Patient type might allow the address to change over time, but it should not allow the id or date of birth to change. This decision can be enforced by the language or simply followed as a design rule.

Taken together, these kinds of types let you match the structure of your code to the structure of the problem. With practice, you will see patterns and decide whether a record, class, enum, or variant is the best fit for you.

4.0 EXAMPLES AND CASE STUDIES

4.1 Simple Examples

To see how user defined types help us model real situations, let us look at a few simple examples. These examples are written in plain language rather than in a specific coding syntax, so that the ideas remain clear and easy to follow. Each example shows how a type brings scattered pieces of data together.

4.1.1 Example Of A Student Type

Suppose we want to build a small system to track students in a course. If we try to use only primitive types, we might have separate arrays for student ids, names, and grades. This is hard to manage and easy to break.

Instead, we define a user defined type named Student with fields such as id, name, email, and grade. Now a single Student value keeps related data together. When we pass a Student to a function that prints a report or updates a grade, we know that all needed data travels as a unit.

4.1.2 Example Of A Book Type

In a library system, we can define a Book type with fields like title, author, publication year, and is available. We might also attach methods such as issue to member and return to shelf if we use classes. Each Book instance then models one real book in the library.

4.2 Case Study: Modeling A Music Library

Now we move to a larger case study. Imagine we are designing a simple MP3 music library application. The goal of the system is to store information about MP3 tracks, show them to the user, and play them on request. User defined types give us a way to model each part of this system in a direct and readable way.

4.2.1 User Defined Types For Mp3 Tracks

We can define a type called Track with fields such as title, artist, album, duration in seconds, and file path. Each Track stands for one MP3 file stored on disk. The file path points to the real file. The other fields store the metadata that a user cares about when they search, sort, or play songs.

We can then define an Artist type with fields like name, country, and a list of tracks. This type lets us group all the tracks for one artist together. A Genre type might also be useful. It could store the name of the genre and a list of tracks that belong to that genre, such as rock, pop, or classical.

4.2.2 Playlists And Library Features

A music library also needs playlists. A Playlist type can have a name and an ordered list of tracks. Methods on Playlist can add a track, remove a track, or move a track up or down. The Player part of the program can then use these methods to move through the playlist in order.

4.3 Case Study: Modeling A Hospital System

Our second case study is a small hospital information system. Here, the key actors are patients, doctors, and appointments. User defined types help us keep each of these ideas clear and separate while also linking them in the right way.

4.3.1 Patient, Doctor, And Appointment Types

We can define a Patient type that stores id, name, date of birth, blood group, and address. A Doctor type may have id, name, specialty, and a list of assigned patients. An Appointment type can connect a patient, a doctor, a date, and a status field that uses an enumeration such as Scheduled, Completed, or Cancelled.

By tying these types together, we get a rich model of the system. When a new appointment is created, the program creates an Appointment value that links to one Patient and one Doctor. When we print a schedule, we loop over Appointment values rather than over separate lists of names and times.

4.4 Common Mistakes In Modeling

Beginners often make a few common mistakes when designing user defined types. One mistake is to use only primitive types, even when a new type would make the code clearer. Another mistake is the opposite: creating too many tiny types that do not add meaning, such as wrapping an integer id in a separate type that is never given more structure.

A further mistake is to mix unrelated ideas in one type. For example, a Student type that also stores details about the last exam room and the cafeteria balance may be carrying too much. In such cases, it is better to create smaller types, such as Exam record and Payment account, and link them to the Student type.

4.5 Reflection Activities

To deepen your understanding, you can try some short reflection activities. Take a real system you know well, such as your college portal, a shopping app, or a music streaming service. On a sheet of paper, list the main nouns that appear on the screen. These may include words like user, order, product, playlist, or course. For each noun, try to imagine a user defined type that could model it inside the code.

Next, write two or three fields that might belong to each type. For a Course type, you might write code, title, and credit hours. For a Playlist type, you might write name, owner, and list of tracks. You do not need to write real code. Simple words are enough at this stage. The goal is to link what you see in the user interface with what might live in the program.

You can also look at a small piece of code from a lab or textbook and ask how it would change if you introduced new types. If the code uses many separate arrays or many parallel lists, try to group the fields into a single record or class. Even this mental exercise helps you build a habit of modeling carefully.

1. User defined types model real world problems inside code.
2. Good type design improves clarity, safety, and reuse.
3. Records, classes, enums, and variants are common user defined types.
4. Types can be mutable or immutable, and this choice should match the real world.
5. Case studies like music libraries and hospital systems show types working together.
6. Careful modeling avoids giant mixed types and tiny empty types.



5.0 Keyword Touch Vocabulary Meaning

Keyword	Simple meaning in this unit
Data type	Rule about a value and how we can use it
Primitive type	Basic built in type like integer or character
User defined type	New type made by the programmer
Record / struct	Group of named fields stored together
Class	Type with data and methods in one unit
Object	Actual instance of a class
Enumeration	Type with a small set of named values
Variant / union	Type whose value can take several shapes
Type system	Rules that connect types to values and expressions
Playlist	Ordered list of tracks in a music library
Appointment	Link between patient, doctor, date, and status